



**Universidad
de Santander**
UDES

ACREDITACIÓN
INTERNACIONAL



EQUAA

Education Quality Accreditation Agency

Patrones de Diseño de Software

Unidad 9: Pruebas Unitarias y de Integración

Post-Contenido 2

Ingeniería de Sistemas

2026

Objetivo de la Actividad

El estudiante implementa pruebas de integración para la capa de persistencia y la capa web de una aplicación Spring Boot, utilizando `@DataJpaTest` y `@WebMvcTest` respectivamente. Adicionalmente, el estudiante configura un pipeline de integración continua con GitHub Actions que ejecuta automáticamente las pruebas y genera un reporte de cobertura con JaCoCo en cada push al repositorio.

Prerrequisitos

- Haber completado Post-Contenido 1 o tener experiencia con Spring Boot + JPA
- JDK 21, Maven 3.9+, IntelliJ IDEA o VS Code con Java Extension Pack
- Cuenta en GitHub con Git configurado (nombre de usuario y email)
- Conocimiento básico de YAML (para configurar el workflow de GitHub Actions)

Descripción del Laboratorio

El laboratorio amplía el microservicio de productos del Post-Contenido 1 (o utiliza uno nuevo) agregando pruebas de integración para el repositorio JPA y el controlador REST, y automatizando su ejecución mediante un pipeline de CI en GitHub Actions con reporte de cobertura JaCoCo.

Paso 1: Configurar el Proyecto con H2 y JaCoCo

El estudiante debe agregar las dependencias necesarias en `pom.xml`. H2 ya viene incluido en `spring-boot-starter-test` para el scope `test`. JaCoCo se configura como plugin de Maven:

```
<!-- En pom.xml - dependencias (H2 ya incluida en starter-test) -->
<dependency>
  <groupId>com.h2database</groupId>
  <artifactId>h2</artifactId>
  <scope>test</scope>
</dependency>

<!-- Plugin JaCoCo para cobertura -->
<plugin>
  <groupId>org.jacoco</groupId>
  <artifactId>jacoco-maven-plugin</artifactId>
  <version>0.8.11</version>
  <executions>
```

```
<execution><goals><goal>prepare-agent</goal></goals></execution>
<execution>
  <id>report</id>
  <phase>test</phase>
  <goals><goal>report</goal></goals>
</execution>
</executions>
</plugin>
```

Crear el archivo `src/test/resources/application-test.properties`:

```
spring.datasource.url=jdbc:h2:mem:testdb;DB_CLOSE_DELAY=-1
spring.datasource.driver-class-name=org.h2.Driver
spring.jpa.hibernate.ddl-auto=create-drop
spring.jpa.show-sql=true
```

Checkpoint de Verificación

- `mvn test` ejecuta sin errores de configuración y genera `target/site/jacoco/index.html`.
- El archivo `application-test.properties` está en `src/test/resources/`.

Paso 2: @DataJpaTest — Pruebas del Repositorio

Las pruebas con `@DataJpaTest` verifican que los métodos del repositorio JPA funcionan correctamente contra H2. Spring Boot configura automáticamente la BD en memoria y revierte cada prueba en una transacción:

```
package com.universidad.productoservice.repository;

import com.universidad.productoservice.domain.Producto;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.orm.jpa.DataJpaTest;

import java.util.List;
import java.util.Optional;

import static org.junit.jupiter.api.Assertions.*;

@DataJpaTest
class ProductoRepositoryTest {

    @Autowired
```

```
private ProductoRepository productoRepository;

@BeforeEach
void setUp() {
    productoRepository.deleteAll();
}

@Test
void save_asignaIdAutomaticamente() {
    Producto guardado = productoRepository
        .save(new Producto(null, "Laptop", 1500.0, 10));
    assertNotNull(guardado.getId());
    assertTrue(guardado.getId() > 0);
}

@Test
void findById_existente_retornaProducto() {
    Producto guardado = productoRepository
        .save(new Producto(null, "Mouse", 50.0, 100));
    Optional<Producto> resultado =
        productoRepository.findById(guardado.getId());
    assertTrue(resultado.isPresent());
    assertEquals("Mouse", resultado.get().getNombre());
}

@Test
void findAll_retornaListaCompleta() {
    productoRepository.save(new Producto(null, "Teclado", 80.0,
50));
    productoRepository.save(new Producto(null, "Monitor", 350.0,
20));
    List<Producto> productos = productoRepository.findAll();
    assertEquals(2, productos.size());
}

@Test
void deleteById_eliminaProducto() {
    Producto guardado = productoRepository
        .save(new Producto(null, "Webcam", 90.0, 15));
    productoRepository.deleteById(guardado.getId());

    assertFalse(productoRepository.findById(guardado.getId()).isPresent());
}
}
```

Checkpoint de Verificación

- Las 4 pruebas de ProductoRepositoryTest pasan en verde.
- @BeforeEach limpia la BD entre pruebas, garantizando aislamiento.
- La prueba findAll retorna exactamente 2 productos tras guardar 2.

Paso 3: @WebMvcTest — Pruebas del Controlador REST

Las pruebas con @WebMvcTest cargan únicamente la capa web. El servicio se provee como @MockBean para controlar las respuestas:

```
package com.universidad.productoservice.controller;

import com.fasterxml.jackson.databind.ObjectMapper;
import com.universidad.productoservice.domain.Producto;
import com.universidad.productoservice.service.ProductoService;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.WebMvcTest;
import org.springframework.boot.test.mock.mockito.MockBean;
import org.springframework.http.MediaType;
import org.springframework.test.web.servlet.MockMvc;

import java.util.List;

import static org.mockito.ArgumentMatchers.*;
import static org.mockito.Mockito.when;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.*;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;

@WebMvcTest(ProductoController.class)
class ProductoControllerTest {

    @Autowired private MockMvc mockMvc;
    @Autowired private ObjectMapper objectMapper;
    @MockBean private ProductoService productoService;

    @Test
    void listarProductos_retorna200ConLista() throws Exception {
        List<Producto> lista = List.of(
            new Producto(1L, "Laptop", 1500.0, 10),
            new Producto(2L, "Mouse", 50.0, 100));
        when(productoService.listarTodos()).thenReturn(lista);

        mockMvc.perform(get("/api/productos")
            .contentType(MediaType.APPLICATION_JSON))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.length()").value(2))
            .andExpect(jsonPath("$[0].nombre").value("Laptop"));
    }
}
```

```
@Test
void crearProducto_datosValidos_retorna201() throws Exception {
    Producto creado = new Producto(1L, "Tablet", 800.0, 5);
    when(productoService.crear(anyString(), anyDouble(), anyInt()))
        .thenReturn(creado);

    String json = """
        {"nombre":"Tablet","precio":800.0,"stock":5}""";

    mockMvc.perform(post("/api/productos")
        .contentType(MediaType.APPLICATION_JSON)
        .content(json))
        .andExpect(status().isCreated())
        .andExpect(jsonPath("$.id").value(1))
        .andExpect(jsonPath("$.nombre").value("Tablet"));
}

@Test
void buscarProducto_noExistente_retorna404() throws Exception {
    when(productoService.buscarPorId(99L))
        .thenReturn(new RuntimeException("Producto no encontrado:
99"));

    mockMvc.perform(get("/api/productos/99"))
        .andExpect(status().isNotFound());
}
}
```

Checkpoint de Verificación

- Las 3 pruebas de ProductoControllerTest pasan en verde.
- La prueba de 404 requiere que el controlador tenga un `@ExceptionHandler` o `@ControllerAdvice` que maneje `RuntimeException` retornando 404.
- mvn test total: todas las pruebas de unidad e integración en verde (≥7 pruebas en total).

Paso 4: Configurar GitHub Actions con JaCoCo

El estudiante debe crear el directorio `.github/workflows/` en la raíz del proyecto y el archivo `ci.yml`:

```
# .github/workflows/ci.yml
name: CI - Pruebas y Cobertura

on:
  push:
    branches: [ main ]
```

```
pull_request:
  branches: [ main ]

jobs:
  build-and-test:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout del repositorio
        uses: actions/checkout@v4

      - name: Configurar JDK 21
        uses: actions/setup-java@v4
        with:
          java-version: "21"
          distribution: "temurin"
          cache: maven

      - name: Compilar y ejecutar todas las pruebas
        run: mvn -B verify --no-transfer-progress

      - name: Subir reporte de cobertura JaCoCo
        uses: actions/upload-artifact@v4
        if: always()
        with:
          name: jacoco-report
          path: target/site/jacoco/
          retention-days: 7
```

Checkpoint de Verificación

- El archivo .github/workflows/ci.yml está en la raíz del proyecto.
- Tras hacer push a main, el workflow aparece en la pestaña Actions de GitHub y ejecuta correctamente.
- El workflow termina con estado "Success" (marca verde) en GitHub.

Paso 5: Verificar Reporte de Cobertura y Documentar

Una vez ejecutado el pipeline, el estudiante debe descargar el artefacto jacoco-report desde GitHub Actions y verificar la cobertura:

- Abrir target/site/jacoco/index.html en el navegador (o el artefacto descargado)
- Verificar que la cobertura de la clase ProductoServiceImpl es $\geq 70\%$ en líneas
- Incluir una captura de pantalla del reporte JaCoCo en el README del proyecto



- Incluir el badge de estado del workflow en el README:

```
# README.md – agregar al inicio:

# Productos Service

![CI] (https://github.com/[usuario]/[repo]/actions/workflows/ci.yml/badge.svg)

## Descripción
Microservicio de gestión de productos con suite completa de pruebas...

## Ejecutar las pruebas
```bash
mvn test # Solo pruebas unitarias
mvn verify # Pruebas + reporte JaCoCo
```

## Cobertura
![Captura JaCoCo] (docs/jacoco-report.png)
```

Checkpoint de Verificación

- El reporte JaCoCo muestra $\geq 70\%$ de cobertura en ProductoServiceImpl.
- El README incluye el badge del workflow y la captura del reporte JaCoCo.
- El repositorio tiene mínimo 3 commits con mensajes descriptivos (setup inicial, pruebas unitarias, CI/CD).
- El pipeline de GitHub Actions ejecuta sin errores y el badge muestra estado verde.

Rúbrica de Evaluación

| Criterio | Excelente
(90–100%) | Bueno (70–
89%) | Aceptable
(50–69%) | Insuficiente
(0–49%) |
|---------------------------------|---|---|--|--|
| Implementación
(40%) | Todas las clases y pruebas implementadas correctamente; la estructura del proyecto sigue las convenciones Java/Maven; | La mayor parte de la implementación es correcta; existen errores menores de configuración o estructura que no impiden la compilación. | La implementación está incompleta o con errores que requieren correcciones; se evidencia comprensión parcial del | No compila o la implementación es incorrecta en sus elementos fundamentales; no evidencia comprensión del laboratorio. |



| Criterio | Excelente
(90–100%) | Bueno (70–
89%) | Aceptable
(50–69%) | Insuficiente
(0–49%) |
|--|--|---|--|---|
| | los mocks están configurados apropiadamente. | | patrón solicitado. | |
| Funcionalidad
(25%) | Todas las pruebas pasan en verde; los casos de error y borde están cubiertos; el reporte JaCoCo muestra $\geq 70\%$ en la capa de negocio. | La mayoría de pruebas pasan; existen uno o dos casos sin cubrir o con assertions incompletas. | Algunas pruebas pasan; la cobertura es baja o el reporte JaCoCo no se genera correctamente. | Las pruebas no pasan o no existen assertions significativas; la funcionalidad no puede verificarse. |
| Documentación
(20%) | README completo con instrucciones de ejecución, descripción del proyecto y evidencia de pruebas en verde (captura de pantalla); commits descriptivos. | README presente con información básica; commits con mensajes adecuados. | README mínimo o con información incompleta; algunos commits sin mensajes descriptivos. | Sin README o documentación; historial de commits sin descripción útil. |
| Estilo y convenciones
(15%) | Nombres descriptivos en métodos de prueba (convención <code>metodoBajoPrueba_condicion_resultado</code>); código limpio sin comentarios innecesarios. | Mayoría de pruebas con nombres adecuados; código ordenado con pocas inconsistencias. | Algunos nombres de prueba son genéricos (<code>test1</code> , <code>testOk</code>); código con inconsistencias de formato. | Nombres no descriptivos o incumplimiento o generalizado de convenciones del lenguaje. |

Entregables

| Elemento | Especificación |
|-------------------|------------------------------|
| Plataforma | GitHub (repositorio público) |



| Elemento | Especificación |
|------------------------|---|
| Nombre del repositorio | apellido-post2-u9 |
| Estructura requerida | README.md + src/ con código fuente + capturas de pantalla de pruebas en verde |
| Commits mínimos | Mínimo 3 commits con mensajes descriptivos que reflejen el progreso del laboratorio |
| Evidencia de ejecución | Captura de pantalla del reporte de pruebas (todas en verde) incluida en el README |